PRAKTIKUM 10

KLASIFIKASI TEKS

A. TUJUAN PEMBELAJARAN

1. Mahasiswa mampu menjelaskan konsep, tujuan, dan tahapan klasifikasi teks.
2. Mahasiswa mampu melakukan prapemrosesan teks, seperti case folding, tokenisasi, penghapusan stopword, dan stemming.
3. Mahasiswa mampu menerapkan algoritma klasifikasi, seperti Naive Bayes, Support Vector Machine, atau Logistic Regression, pada data teks.
4. Mahasiswa mampu mengevaluasi dan menginterpretasikan kinerja model menggunakan confusion matrix, accuracy, precision, recall, dan F1-score.

B. DASAR TEORI

Klasifikasi teks merupakan salah satu teknik dalam text mining yang digunakan untuk mengelompokkan dokumen atau data teks ke dalam kategori tertentu berdasarkan isi dan karakteristiknya. Teknik ini termasuk dalam supervised learning karena model dibangun menggunakan data latih yang telah memiliki label kelas. Melalui proses pelatihan tersebut, model mempelajari pola hubungan antara kata atau fitur dalam teks dan kategori yang telah ditentukan. Setelah proses pelatihan selesai, model dapat digunakan untuk memprediksi kategori dari teks baru yang belum pernah diproses sebelumnya. Klasifikasi teks banyak diterapkan pada analisis sentimen, penyaringan spam, pengelompokan berita, identifikasi topik, dan deteksi ujaran kebencian.

Sebelum diklasifikasikan, data teks perlu melalui tahap prapemrosesan agar menjadi lebih terstruktur dan mudah dianalisis. Tahapan ini umumnya meliputi case folding untuk menyeragamkan penggunaan huruf, tokenisasi untuk memisahkan teks menjadi kata atau token, penghapusan stopword untuk menghilangkan kata-kata yang kurang informatif, serta stemming atau lemmatisasi untuk mengubah kata ke bentuk dasarnya. Prapemrosesan dapat mengurangi variasi dan jumlah fitur yang tidak relevan sehingga model dapat mempelajari informasi penting secara lebih efektif.

Teks yang telah melalui tahap prapemrosesan selanjutnya diubah menjadi representasi numerik karena algoritma pembelajaran mesin tidak dapat mengolah teks secara langsung. Metode representasi yang umum digunakan antara lain Bag of Words dan Term Frequency–Inverse Document Frequency (TF-IDF). Bag of Words merepresentasikan dokumen berdasarkan frekuensi kemunculan kata, sedangkan TF-IDF memberikan bobot lebih besar kepada kata yang penting dalam suatu dokumen tetapi tidak terlalu sering muncul di seluruh kumpulan dokumen. Selain itu, teks dapat direpresentasikan menggunakan word embedding, seperti Word2Vec, GloVe, atau representasi kontekstual berbasis transformer, yang mampu menangkap hubungan semantik antarkata.

Beberapa algoritma yang dapat digunakan dalam klasifikasi teks adalah Naive Bayes, Logistic Regression, Support Vector Machine, Decision Tree, Random Forest, serta metode deep learning. Naive Bayes banyak digunakan karena sederhana, cepat, dan efektif untuk data teks berdimensi tinggi. Support Vector Machine bekerja dengan mencari batas pemisah terbaik antarkelas, sedangkan Logistic Regression memperkirakan probabilitas suatu teks termasuk dalam kelas tertentu. Pemilihan algoritma perlu mempertimbangkan ukuran data, jumlah kategori, karakteristik fitur, kebutuhan interpretasi, dan sumber daya komputasi yang tersedia.

Kinerja model klasifikasi teks dapat dinilai menggunakan confusion matrix dan sejumlah metrik evaluasi, seperti accuracy, precision, recall, dan F1-score. Accuracy menunjukkan proporsi seluruh prediksi yang benar, sedangkan precision mengukur ketepatan model dalam memprediksi suatu kelas. Recall menunjukkan kemampuan model dalam menemukan seluruh data yang seharusnya termasuk dalam kelas tersebut. F1-score merupakan rata-rata harmonis antara precision dan recall sehingga sesuai digunakan ketika distribusi kelas tidak seimbang. Evaluasi yang tepat diperlukan untuk memastikan bahwa model tidak hanya memiliki nilai akurasi tinggi, tetapi juga mampu melakukan klasifikasi secara konsisten dan sesuai dengan tujuan penerapannya.

Keberhasilan klasifikasi teks dipengaruhi oleh kualitas data, ketepatan pelabelan, metode prapemrosesan, representasi fitur, dan algoritma yang digunakan. Data yang tidak seimbang, bahasa informal, kesalahan penulisan, ambiguitas kata, serta penggunaan konteks dan sarkasme merupakan beberapa tantangan dalam proses klasifikasi. Oleh karena itu, pengembangan model klasifikasi teks perlu dilakukan secara sistematis, mulai dari pengumpulan dan pembersihan data, pembentukan fitur, pelatihan model, evaluasi, hingga penyempurnaan model agar diperoleh hasil yang akurat dan dapat digunakan sesuai kebutuhan..

C. LANGKAH PRAKTIKUM

1. Buka Google Colab.
2. Pilih **New Notebook** untuk membuat notebook baru.
3. Tambahkan satu sel kode.
4. Salin seluruh kode pada bagian **Source Code** ke dalam satu sel tersebut.
5. Jalankan kode dengan menekan tombol **Run** atau menggunakan kombinasi tombol `Shift + Enter`.
6. Amati data, hasil prapemrosesan, nilai evaluasi, confusion matrix, dan hasil prediksi teks baru.
7. Ubah contoh teks pada bagian pengujian untuk melihat perubahan hasil klasifikasi.

```
# ============================================================
# PRAKTIKUM KLASIFIKASI TEKS
# Studi Kasus: Klasifikasi Sentimen Bahasa Indonesia
# Metode: TF-IDF dan Logistic Regression
# ============================================================

# 1. IMPORT LIBRARY
import re
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split,
cross_val_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)


# 2. MEMBUAT DATASET
data = {
    "teks": [
        # Kelas positif
        "Pelayanan di toko ini sangat baik dan ramah",
        "Saya sangat puas dengan kualitas produk ini",
        "Aplikasi ini mudah digunakan dan sangat membantu",
        "Pengiriman barang berlangsung cepat dan aman",
        "Makanan di restoran ini sangat enak",
        "Fitur terbaru aplikasi ini sangat bermanfaat",
        "Petugas melayani pelanggan dengan sopan",
        "Produk yang saya terima sesuai dengan deskripsi",
        "Saya senang menggunakan layanan perusahaan ini",
        "Harga produknya terjangkau dan kualitasnya bagus",
        "Proses pembayaran sangat mudah dan cepat",
        "Pengalaman berbelanja di sini sangat menyenangkan",

        # Kelas negatif
        "Pelayanan di toko ini sangat buruk",
        "Saya kecewa dengan kualitas produk ini",
        "Aplikasi ini sulit digunakan dan sering bermasalah",
        "Pengiriman barang sangat lambat",
        "Makanan yang disajikan tidak enak",
        "Fitur terbaru justru membuat aplikasi lambat",
        "Petugas tidak ramah kepada pelanggan",
        "Produk yang diterima tidak sesuai deskripsi",
        "Saya menyesal menggunakan layanan ini",
        "Harga produk terlalu mahal untuk kualitas seperti ini",
        "Proses pembayaran sering mengalami kegagalan",
        "Pengalaman berbelanja di sini sangat mengecewakan",

        # Kelas netral
        "Toko tersebut buka setiap hari pukul delapan",
```

```
        "Produk tersedia dalam tiga pilihan warna",
        "Aplikasi ini dapat diunduh melalui Play Store",
        "Barang dikirim menggunakan jasa pengiriman reguler",
        "Restoran menyediakan menu makanan Indonesia",
        "Fitur terbaru dirilis pada bulan ini",
        "Petugas berada di meja layanan pelanggan",
        "Produk tersebut memiliki berat lima ratus gram",
        "Layanan dapat digunakan melalui situs resmi",
        "Harga produk tercantum pada halaman katalog",
        "Pembayaran dapat dilakukan melalui transfer bank",
        "Toko tersebut berada di pusat kota"
    ],
    "label": (
        ["positif"] * 12 +
        ["negatif"] * 12 +
        ["netral"] * 12
    )
}

df = pd.DataFrame(data)

print("=" * 60)
print("LIMA DATA PERTAMA")
print("=" * 60)
print(df.head())

print("\nJumlah data:", len(df))

print("\nDistribusi kelas:")
print(df["label"].value_counts())


# 3. MEMBUAT FUNGSI PRAPEMROSESAN TEKS
def bersihkan_teks(teks):
    teks = teks.lower()                          # Mengubah teks
menjadi huruf kecil
    teks = re.sub(r"http\S+|www\S+", " ", teks) # Menghapus alamat
situs
    teks = re.sub(r"@\w+|#\w+", " ", teks)      # Menghapus
mention dan hashtag
    teks = re.sub(r"[^a-zA-Z\s]", " ", teks)    # Menghapus angka
dan tanda baca
    teks = re.sub(r"\s+", " ", teks).strip()    # Menghapus spasi
berlebih
    return teks


df["teks_bersih"] = df["teks"].apply(bersihkan_teks)

print("\n" + "=" * 60)
print("CONTOH HASIL PRAPEMROSESAN")
print("=" * 60)
print(df[["teks", "teks_bersih"]].head())


# 4. MENENTUKAN FITUR DAN LABEL
X = df["teks_bersih"]
```

```
y = df["label"]


# 5. MEMBAGI DATA MENJADI DATA LATIH DAN DATA UJI
X_latih, X_uji, y_latih, y_uji = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

print("\n" + "=" * 60)
print("PEMBAGIAN DATA")
print("=" * 60)
print("Jumlah data latih:", len(X_latih))
print("Jumlah data uji  :", len(X_uji))


# 6. MEMBUAT PIPELINE TF-IDF DAN LOGISTIC REGRESSION
model = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            ngram_range=(1, 2),
            min_df=1
        )
    ),
    (
        "klasifikasi",
        LogisticRegression(
            max_iter=1000,
            random_state=42
        )
    )
])


# 7. MELATIH MODEL
model.fit(X_latih, y_latih)


# 8. MELAKUKAN PREDIKSI TERHADAP DATA UJI
y_prediksi = model.predict(X_uji)


# 9. MENGEVALUASI MODEL
akurasi = accuracy_score(y_uji, y_prediksi)
kelas = ["positif", "negatif", "netral"]

print("\n" + "=" * 60)
print("HASIL EVALUASI MODEL")
print("=" * 60)
print(f"Akurasi: {akurasi:.2%}")

print("\nClassification report:")
print(
```

```
    classification_report(
        y_uji,
        y_prediksi,
        labels=kelas,
        zero_division=0
    )
)


# 10. MENAMPILKAN HASIL PREDIKSI DATA UJI
hasil_uji = pd.DataFrame({
    "teks": X_uji,
    "label_sebenarnya": y_uji,
    "hasil_prediksi": y_prediksi
}).reset_index(drop=True)

print("\n" + "=" * 60)
print("PERBANDINGAN LABEL SEBENARNYA DAN HASIL PREDIKSI")
print("=" * 60)
print(hasil_uji.to_string(index=False))


# 11. MEMBUAT CONFUSION MATRIX
cm = confusion_matrix(
    y_uji,
    y_prediksi,
    labels=kelas
)

plt.figure(figsize=(7, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=kelas,
    yticklabels=kelas
)

plt.title("Confusion Matrix Klasifikasi Sentimen")
plt.xlabel("Label Prediksi")
plt.ylabel("Label Sebenarnya")
plt.tight_layout()
plt.show()


# 12. MELAKUKAN CROSS-VALIDATION
skor_cv = cross_val_score(
    model,
    X,
    y,
    cv=3,
    scoring="accuracy"
)

print("\n" + "=" * 60)
print("HASIL CROSS-VALIDATION")
```

```
print("=" * 60)
print("Skor setiap fold:", skor_cv)
print(f"Rata-rata skor  : {skor_cv.mean():.2%}")


# 13. MENGUJI MODEL DENGAN TEKS BARU
teks_baru = [
    "Saya sangat senang karena pelayanannya cepat",
    "Produknya rusak dan saya sangat kecewa",
    "Toko akan dibuka pada pukul sembilan pagi",
    "Aplikasi ini bagus tetapi terkadang berjalan lambat"
]

teks_baru_bersih = [bersihkan_teks(teks) for teks in teks_baru]

prediksi_baru = model.predict(teks_baru_bersih)
probabilitas_baru = model.predict_proba(teks_baru_bersih)
nama_kelas = model.named_steps["klasifikasi"].classes_

hasil_baru = pd.DataFrame({
    "teks_baru": teks_baru,
    "hasil_prediksi": prediksi_baru,
    "tingkat_keyakinan": probabilitas_baru.max(axis=1)
})

hasil_baru["tingkat_keyakinan"] = hasil_baru[
    "tingkat_keyakinan"
].map(lambda nilai: f"{nilai:.2%}")

print("\n" + "=" * 60)
print("HASIL KLASIFIKASI TEKS BARU")
print("=" * 60)
print(hasil_baru.to_string(index=False))


# 14. MENAMPILKAN PROBABILITAS SETIAP KELAS
tabel_probabilitas = pd.DataFrame(
    probabilitas_baru,
    columns=nama_kelas
)

tabel_probabilitas.insert(0, "teks_baru", teks_baru)

print("\n" + "=" * 60)
print("PROBABILITAS SETIAP KELAS")
print("=" * 60)
print(tabel_probabilitas.to_string(index=False))
```

Penjelasan Code:

1. Import library

Program mengimpor beberapa library yang diperlukan. pandas digunakan untuk mengelola dataset dalam bentuk tabel. re digunakan untuk membersihkan teks dengan pola regular expression. matplotlib dan seaborn digunakan untuk membuat visualisasi confusion matrix. Sementara itu, beberapa modul dari scikit-learn digunakan untuk membagi data, membentuk fitur TF-IDF, membangun model, dan melakukan evaluasi.

2. Pembuatan dataset

Dataset terdiri atas dua kolom, yaitu teks dan label. Kolom teks berisi kalimat yang akan dianalisis, sedangkan kolom label berisi kategori sentimen dari setiap kalimat. Dalam praktikum ini terdapat tiga kategori, yaitu positif, negatif, dan netral. Dataset dibuat langsung di dalam program agar praktikum dapat dijalankan tanpa mengunggah berkas tambahan.

3. Prapemrosesan teks

Fungsi bersihkan_teks() digunakan untuk membersihkan data sebelum diproses oleh model. Prapemrosesan yang dilakukan mencakup:

- Mengubah seluruh huruf menjadi huruf kecil.
- Menghapus alamat situs.
- Menghapus mention dan hashtag.
- Menghapus angka serta tanda baca.
- Menghapus spasi yang berlebihan.

Prapemrosesan membantu mengurangi variasi penulisan yang tidak diperlukan. Sebagai contoh, kata “Produk” dan “produk” akan dianggap sebagai kata yang sama setelah keduanya diubah menjadi huruf kecil.

4. Penentuan fitur dan label

Variabel X berisi teks yang telah dibersihkan dan berperan sebagai fitur atau masukan model. Variabel y berisi label sentimen yang akan dipelajari dan diprediksi oleh model.

5. Pembagian data

Dataset dibagi menjadi data latih dan data uji. Sebanyak 75% data digunakan untuk melatih model, sedangkan 25% sisanya digunakan untuk menguji kemampuan model pada data yang tidak digunakan selama pelatihan.

Parameter stratify=y digunakan agar perbandingan jumlah kelas positif, negatif, dan netral tetap seimbang pada data latih dan data uji. Parameter random_state=42 digunakan agar pembagian data menghasilkan susunan yang sama setiap kali program dijalankan.

6. Pembentukan fitur TF-IDF

Algoritma pembelajaran mesin tidak dapat mengolah teks secara langsung. Oleh karena itu, teks diubah menjadi angka menggunakan metode TF-IDF. Metode ini memberikan bobot kepada setiap kata berdasarkan tingkat kepentingannya dalam suatu dokumen dan keseluruhan dataset.

Parameter ngram_range=(1, 2) membuat TF-IDF mempelajari kata tunggal dan pasangan kata. Sebagai contoh, model tidak hanya mempelajari kata “sangat” dan “baik” secara terpisah, tetapi juga pasangan kata “sangat baik”.

7. Pembuatan model Logistic Regression

Logistic Regression merupakan algoritma supervised learning yang dapat digunakan untuk memprediksi kategori suatu data. Dalam praktikum ini, model mempelajari hubungan antara bobot kata yang dihasilkan TF-IDF dan kelas sentimen positif, negatif, atau netral.

TF-IDF dan Logistic Regression digabungkan menggunakan Pipeline. Dengan demikian, proses transformasi teks dan klasifikasi dapat dijalankan secara berurutan dalam satu model.

8. Pelatihan model

Perintah model.fit(X_latih, y_latih) digunakan untuk melatih model. Pada tahap ini, model mempelajari pola kata yang sering muncul pada setiap kategori. Sebagai contoh, kata “baik”, “senang”, dan “puas” dapat menjadi petunjuk untuk kelas positif, sedangkan kata “buruk”, “kecewa”, dan “lambat” dapat menjadi petunjuk untuk kelas negatif.

9. Prediksi data uji

Setelah proses pelatihan selesai, model digunakan untuk memprediksi label data uji. Hasil prediksi kemudian dibandingkan dengan label sebenarnya untuk mengetahui kemampuan model dalam mengklasifikasikan teks.

10. Evaluasi model

Model dievaluasi menggunakan beberapa metrik berikut:

- **Accuracy** menunjukkan persentase seluruh prediksi yang benar.
- **Precision** menunjukkan ketepatan model ketika memprediksi suatu kelas.

- **Recall** menunjukkan kemampuan model dalam menemukan seluruh data dari suatu kelas.
- **F1-score** menunjukkan keseimbangan antara precision dan recall.
- **Support** menunjukkan jumlah data uji pada setiap kelas.

Nilai evaluasi pada praktikum dapat berubah apabila isi dataset, pembagian data, atau parameter model diubah.

11. Confusion matrix

Confusion matrix memperlihatkan perbandingan antara label sebenarnya dan label hasil prediksi. Baris menunjukkan label sebenarnya, sedangkan kolom menunjukkan label yang diprediksi model. Angka pada diagonal utama menunjukkan prediksi yang benar. Angka di luar diagonal menunjukkan kesalahan klasifikasi.

12. Cross-validation

Cross-validation digunakan untuk memperoleh evaluasi yang lebih stabil. Dataset dibagi menjadi tiga bagian atau fold. Model dilatih dan diuji sebanyak tiga kali dengan bagian data yang berbeda. Rata-rata skor dari seluruh fold dapat memberikan gambaran performa model yang lebih baik dibandingkan hanya menggunakan satu pembagian data.

13. Pengujian teks baru

Model yang telah dilatih digunakan untuk mengklasifikasikan beberapa kalimat baru. Sebelum diprediksi, kalimat baru dibersihkan menggunakan fungsi prapemrosesan yang sama. Model kemudian menghasilkan label prediksi dan tingkat keyakinannya.

14. Probabilitas setiap kelas

Fungsi predict_proba() menghasilkan probabilitas untuk setiap kelas. Model memilih kelas yang memiliki probabilitas paling tinggi sebagai hasil prediksi. Probabilitas tidak selalu menunjukkan bahwa prediksi benar, tetapi memperlihatkan tingkat keyakinan model berdasarkan pola yang dipelajari.

D. ANALISIS HASIL

Setelah program dijalankan, lakukan analisis dengan menjawab pertanyaan berikut:

1. Berapa nilai accuracy yang dihasilkan oleh model?
2. Kelas manakah yang memiliki nilai precision paling tinggi?
3. Kelas manakah yang memiliki nilai recall paling rendah?
4. Berapa banyak teks yang salah diklasifikasikan berdasarkan confusion matrix?
5. Apakah rata-rata hasil cross-validation berbeda jauh dari accuracy data uji?
6. Bagaimana hasil klasifikasi untuk empat teks baru?

7. Mengapa kalimat yang mengandung kata positif dan negatif sekaligus lebih sulit diklasifikasikan?
8. Apa pengaruh ukuran dan keragaman dataset terhadap hasil klasifikasi?

E. TUGAS

1. Tambahkan minimal 10 teks baru untuk setiap kelas.
2. Ubah perbandingan data latih dan data uji menjadi 80:20.
3. Bandingkan penggunaan `ngram_range=(1, 1)` dan `ngram_range=(1, 2).`
4. Ganti algoritma Logistic Regression dengan Multinomial Naive Bayes.
5. Buat minimal lima kalimat baru dan analisis hasil prediksinya.
6. Catat perubahan accuracy, precision, recall, dan F1-score setelah dataset ditambah.
7. Buat kesimpulan mengenai algoritma atau pengaturan yang memberikan hasil terbaik.

F. KESIMPULAN PRAKTIKUM

Pada praktikum ini, klasifikasi teks dilakukan melalui beberapa tahap, yaitu menyiapkan dataset, membersihkan teks, mengubah teks menjadi representasi numerik menggunakan TF-IDF, melatih model Logistic Regression, dan mengevaluasi hasilnya. Model yang telah dilatih dapat digunakan untuk memprediksi sentimen dari teks baru. Namun, hasil model pada contoh ini belum dapat mewakili penggunaan di dunia nyata karena dataset masih berukuran kecil dan sederhana. Model yang akan diterapkan pada kondisi nyata memerlukan data yang lebih banyak, beragam, seimbang, dan memiliki label yang akurat.